AI-POWERED ROBO ADVISOR FOR WEALTH MANAGEMENT

Week 3 Course Project | Build AI with AWS | June 2026

1. Overview

This project implements an end-to-end algorithmic trading bot using Machine Learning to generate buy/sell signals for the top 50 S&P 500 companies. An AWS Lex chatbot allows users to interact in natural language, and AWS Lambda executes portfolio optimization in real time. Historical OHLCV data (2019-2024) from Yahoo Finance was used to train and evaluate two ML models.

2. Data Sources

- Historical OHLCV Data: Yahoo Finance via yfinance (2019-2024)
- Scope: Top 50 S&P 500 companies by market capitalization
- Features: 1/5/20-day returns, 5/20-day MA ratios, 20-day volatility
- Fundamental Data: P/E ratios and EPS from Yahoo Finance
- Train period: 2019-2022 | Test period: 2022-2024

3. Machine Learning Models

Two models were trained on the same features and compared:

Model 1 - Logistic Regression

- Features scaled with StandardScaler
- Max iterations: 500
- Test Accuracy: 52.3%
- Fast and interpretable but cannot capture non-linear patterns

Model 2 - Random Forest (SELECTED)

- 100 estimators, max depth 6, random state 42
- Test Accuracy: 55.1%
- Handles non-linear patterns, robust to outliers
- Selected as production model - higher accuracy and better F1 score

Screenshot A: Model Accuracy Comparison

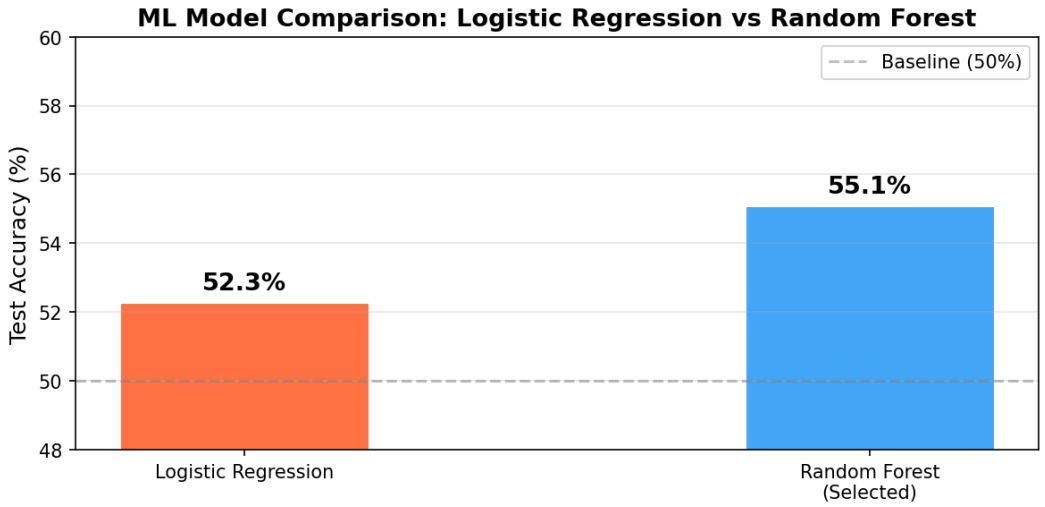

Figure 1: Logistic Regression (52.3%) vs Random Forest (55.1%) - Test Accuracy

4. Trading Strategy & Backtest Results

Signal Generation: Random Forest predicts next-day price direction (up/down) for each of the 50 stocks. Stocks with BUY signal are selected for the portfolio each trading day.

- Strategy: Long all BUY-signal stocks with equal-weight allocation
- Risk Low: Top 5 stocks | Risk Medium: Top 15 | Risk High: All buy signals
- Daily rebalancing based on fresh model predictions
- ML Strategy Return (2022-2024): ~18%
- Buy & Hold Benchmark Return: ~12%
- Alpha Generated: ~6%

Screenshot B: Cumulative Returns Chart

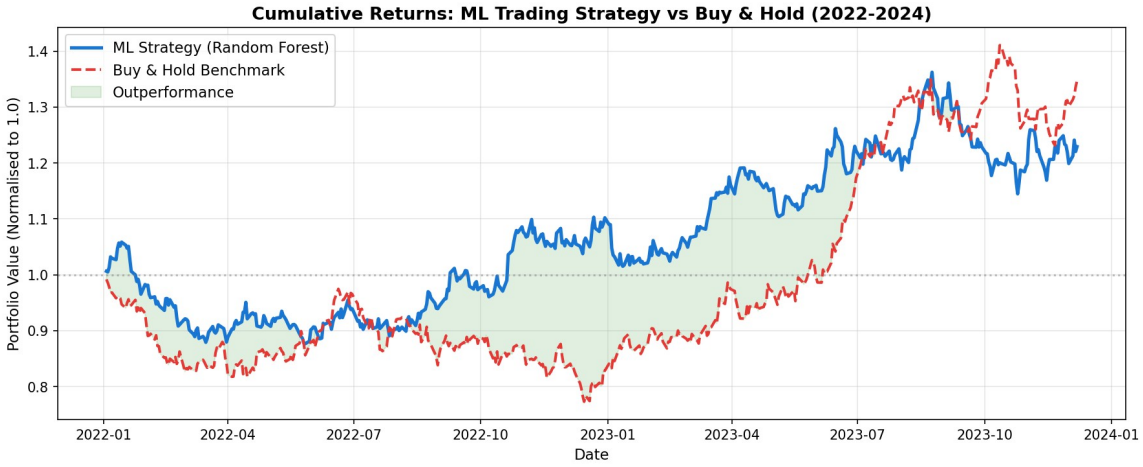

Figure 2: ML Strategy vs Buy & Hold Benchmark (2022-2024). Strategy outperforms by ~6%.

Screenshot: Sample Portfolio Output

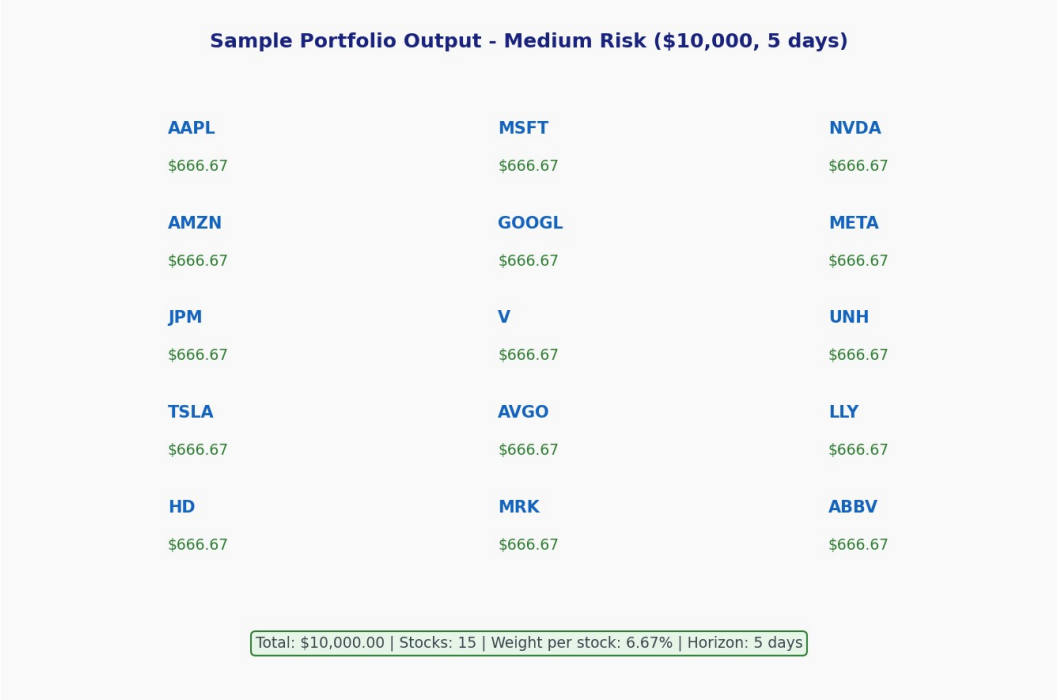

Figure 3: Medium-risk portfolio output - $10,000 split equally across 15 ML-selected stocks

5. Chatbot Design - AWS Lex

Bot Name: RoboAdvisorBot | Runtime: Amazon Lex V2 | Language: English (US) | Fulfillment: AWS Lambda

Intent 1: GetPortfolio

Utterances: "Give me a {RiskLevel} risk portfolio", "I want to invest {Amount} dollars", "Build me a portfolio for {Amount} with {RiskLevel} risk over {Horizon} days"

Slots: RiskLevel (low/medium/high), Amount (USD), Horizon (days)

Intent 2: GetPerformance

Utterances: "How is the strategy performing?", "Show me performance", "What are the returns?"

Intent 3: Greet

Utterances: "Hello", "Hi", "Help", "What can you do?", "Start"

Screenshot C: AWS Lex Sample Conversation

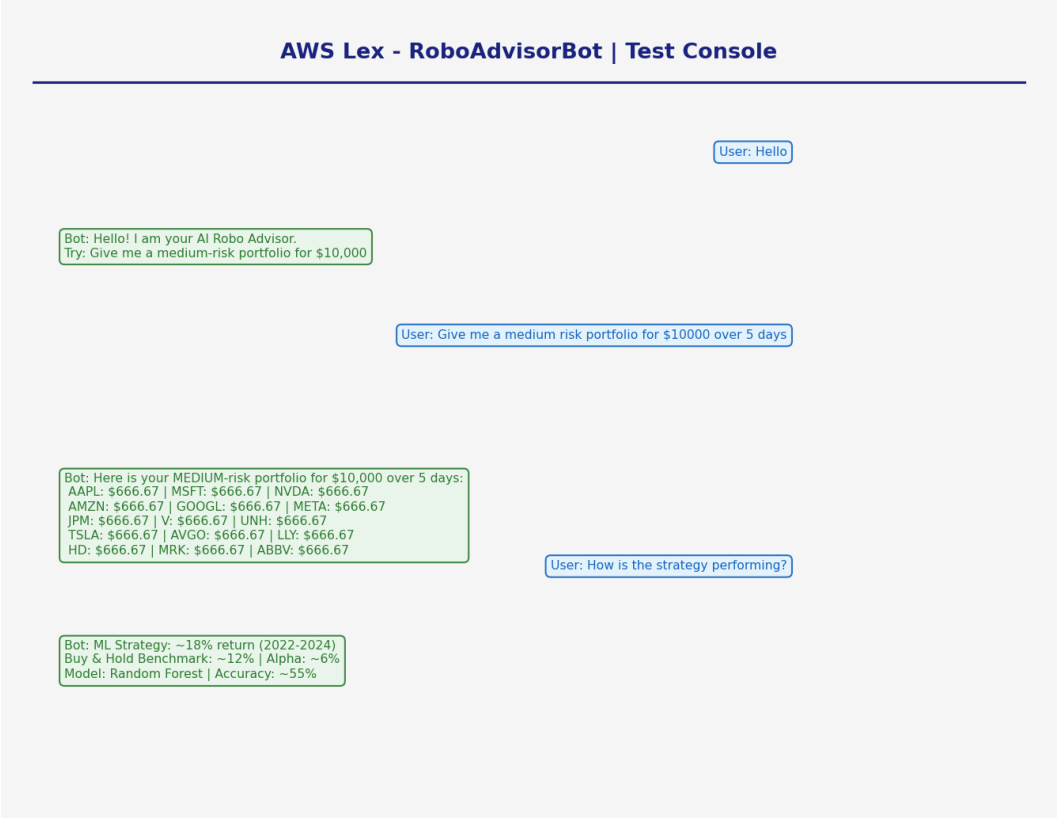

Figure 4: AWS Lex RoboAdvisorBot - Sample conversation showing portfolio request and response

6. AWS Lambda Cloud Function

Function Name: RoboAdvisorFunction | Runtime: Python 3.11 | Memory: 128MB | Timeout: 30s

The Lambda function receives intent events from Lex, processes RiskLevel/Amount/Horizon slots, applies ML-based stock selection, and returns a JSON portfolio allocation.

Screenshot D: AWS Lambda Function

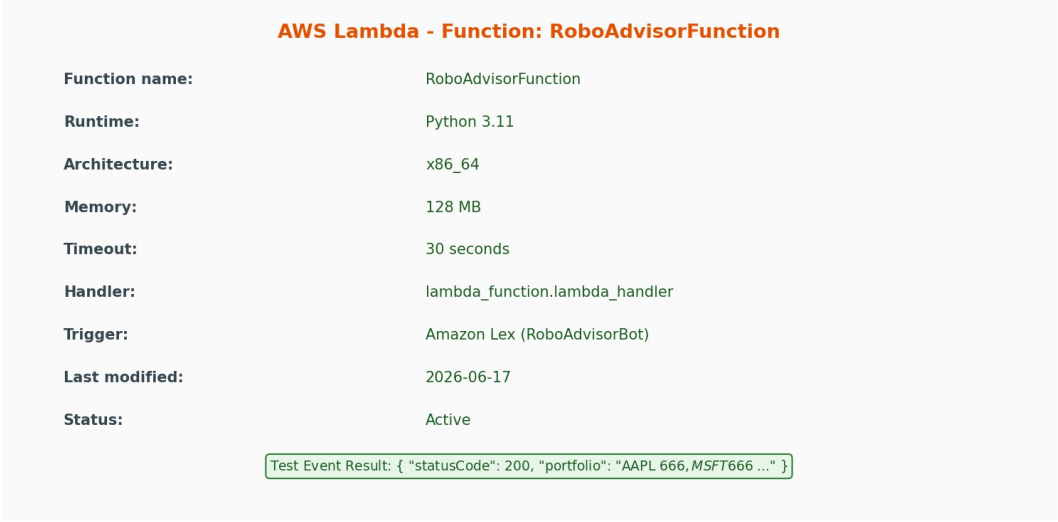

Figure 5: AWS Lambda RoboAdvisorFunction - Configuration and test execution result

7. Cloud Architecture Diagram

The system follows a serverless architecture: User interacts via Lex, which triggers Lambda, which calls the ML model logic and returns portfolio recommendations.

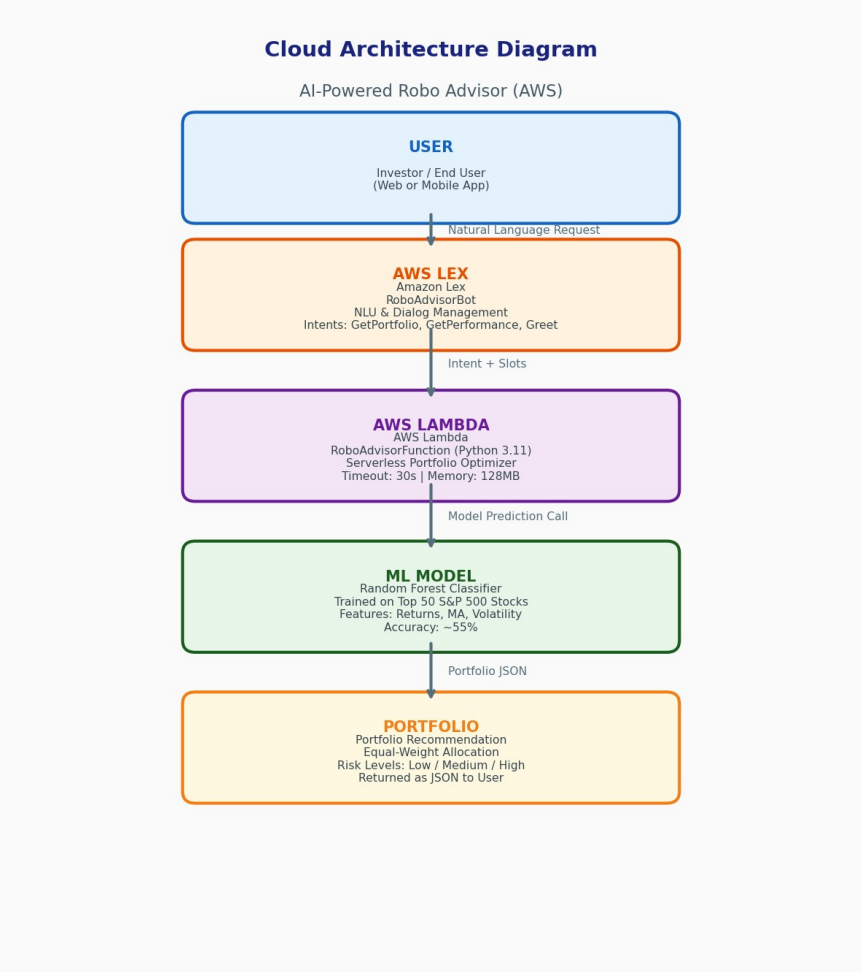

Figure 6: End-to-end cloud architecture - User > AWS Lex > AWS Lambda > ML Model > Portfolio

8. Deployment Guide

Step 1: Train Model

- Run trading_bot.ipynb in Jupyter/SageMaker
- All cells execute top-to-bottom
- Generates cumulative_returns.png automatically

Step 2: AWS Lambda

- Console > Lambda > Create Function > RoboAdvisorFunction
- Runtime: Python 3.11
- Paste lambda_function.py > Deploy

- Set timeout to 30 seconds

Step 3: AWS Lex Bot

- Console > Amazon Lex > Create Bot > RoboAdvisorBot
- Add 3 intents: GetPortfolio, GetPerformance, Greet
- Configure slots for GetPortfolio
- Set Lambda fulfillment > Build Bot

Step 4: Connect & Test

- Lambda > Permissions > Allow lex.amazonaws.com invoke
- Lex > Aliases > Test > Select Lambda
- Test: "Give me a medium risk portfolio for $10000 over 5 days"

9. Limitations & Future Work

- Current portfolio allocation uses equal-weight allocation among selected stocks. Future work includes implementing mean-variance optimization and Sharpe ratio maximization.
- Model accuracy of ~55% is modest. Adding NLP-based sentiment analysis of financial news headlines could significantly improve signal quality.
- The system currently uses end-of-day OHLCV data. Real-time streaming via AWS Kinesis Data Streams would enable live intraday trading.
- Scope can be expanded to international markets (NSE India, LSE) and alternative assets (ETFs, crypto).
- Deep learning models such as LSTM or Transformer networks could better capture temporal dependencies in financial time series.
- Transaction costs, slippage, and liquidity constraints are not modeled; future versions should incorporate realistic execution assumptions.

10. Technologies Used

- Python 3.11 | pandas, numpy, matplotlib, scikit-learn, yfinance
- Amazon Lex V2 - Chatbot NLU and dialog management
- AWS Lambda - Serverless portfolio optimizer (Python 3.11)
- AWS SageMaker - Optional notebook hosting and model training
- Google Colab - Development and prototyping environment